

Documentación Técnica

Proyecto: Sistema de Gestión y Distribución de
Ítems de Trabajo

Autor: David Carrera Castro

Backend: ASP.NET Core + Entity Framework
Core + SQL Server

Julio 2026

Contenido

1. Objetivo	3
2. Tecnologías utilizadas	3
3. Arquitectura.....	3
4. Flujo de la aplicación.....	3
5. Modelo de datos	4
6. Reglas de negocio.....	4
7. Algoritmo de asignación.....	4
8. Endpoints.....	5
9. Ejemplo de creación.....	5
10. Cómo ejecutar.....	5
11. Decisiones de diseño	5
12. Mejoras futuras	5
13. Conclusión.....	6

1. Objetivo

Desarrollar una API REST para gestionar usuarios e ítems de trabajo, implementando reglas automáticas de distribución basadas en prioridad, fecha de entrega y carga de trabajo de los usuarios.

2. Tecnologías utilizadas

- ASP.NET Core
 - C#
 - Entity Framework Core
 - SQL Server
 - Swagger/OpenAPI
 - Git/GitHub

3. Arquitectura

Se implementó una arquitectura por capas para separar responsabilidades:

Controllers: reciben las solicitudes HTTP.

Services: implementan la lógica de negocio.

Repositories: encapsulan el acceso a datos.

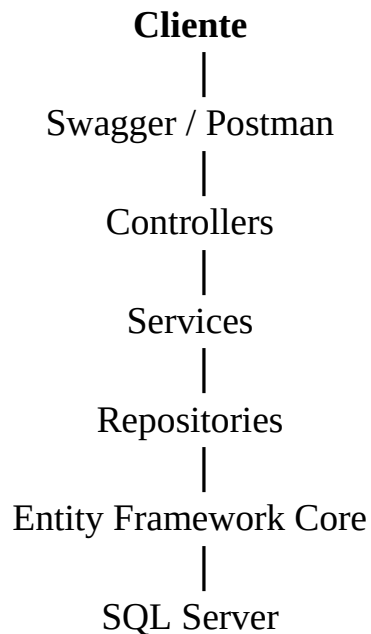
Entities: representan las tablas.

DTOs: modelos de entrada/salida.

Data: DbContext y configuración.

Migrations: versionado de la base de datos.

4. Flujo de la aplicación



5. Modelo de datos

AppUser

- Id
- Username

WorkItem

- Id
- Title
- Description
- DueDate
- Priority
- Status
- AppUserId(FK)

Relación: Un usuario puede tener múltiples WorkItems (1:N).

6. Reglas de negocio

- Si la fecha de entrega es menor a tres días, el WorkItem se asigna al usuario con menor cantidad de ítems.
- Si la prioridad es alta, se asigna al usuario con menor cantidad de pendientes.
- Un usuario con más de tres pendientes de prioridad alta se considera saturado y no participa en la asignación.
- Los pendientes se consultan ordenados por prioridad y fecha de vencimiento.

7. Algoritmo de asignación

Inicio

↓

¿Fecha urgente (<3 días)?

├ Sí → Usuario con menos ítems

└ No → ¿Prioridad alta?

├ Sí → Excluir saturados → Usuario con menos pendientes

└ No → Usuario con menos pendientes

↓

Guardar WorkItem

8. Endpoints

Método	Ruta	Descripción
GET	/api/users	Obtiene todos los usuarios
GET	/api/users/{id}	Obtiene un usuario por Id
POST	/api/users	Crea un usuario
GET	/api/workitems	Obtiene todos los WorkItems
GET	/api/workitems/{id}	Obtiene un WorkItem por Id
POST	/api/workitems	Crea y asigna automáticamente un WorkItem

9. Ejemplo de creación

POST /api/workitems

```
{
  "title": "Incidencia crítica",
  "description": "Validar asignación",
  "dueDate": "2026-07-10T10:00:00",
  "priority": 2
}
```

10. Cómo ejecutar

1. Restaurar paquetes NuGet.
2. Ejecutar Update-Database.
3. Ejecutar la API.
4. Abrir Swagger.
5. Probar los endpoints.

11. Decisiones de diseño

Se utilizó Repository Pattern para desacoplar el acceso a datos y Service Layer para concentrar la lógica de negocio. Entity Framework Core administra el acceso a SQL Server mediante migraciones.

12. Mejoras futuras

- Separar Usuarios y WorkItems en microservicios independientes.
- Implementar autenticación.
- Agregar pruebas unitarias e integración.
- Incorporar logging y manejo centralizado de excepciones.
- Integrar SonarQube y CI/CD.

13. Conclusión

El proyecto cumple los requerimientos funcionales de la prueba técnica implementando una solución mantenible, escalable y organizada por capas.